SE LAB-1 SUBMISSION
NAME: S N Niharika
SRN: PES2UG24CS414

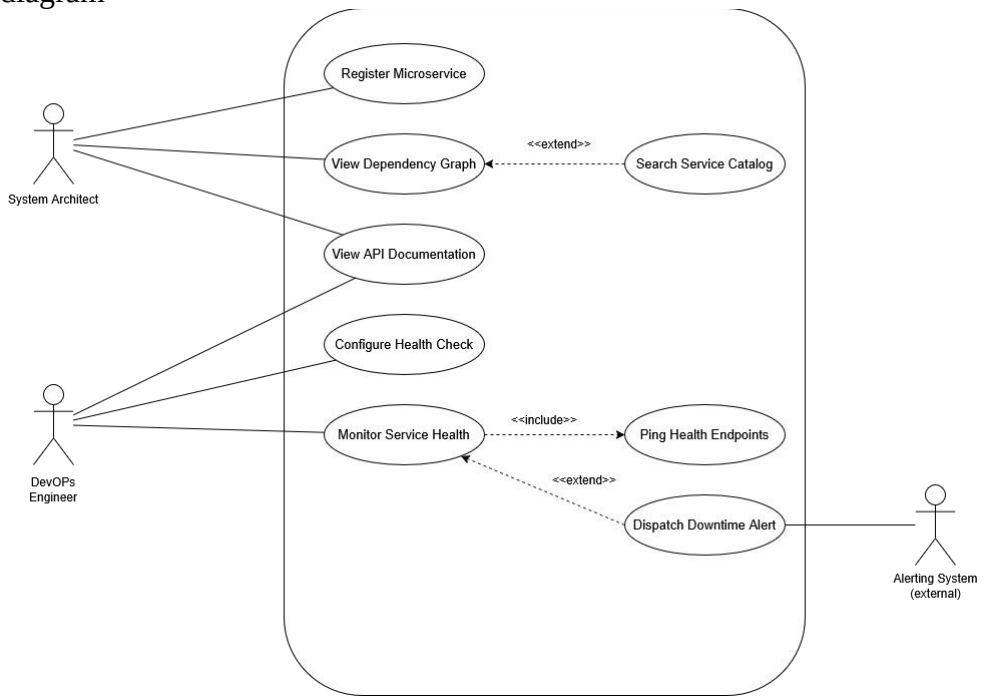
PROBLEM STATEMENT 42
INTERNAL MICROSERVICE CATALOG AND HEALTH PORTAL

1. Requirements table

REQ ID	TYPE	DESCRIPTION	PRIORITY	ACCEPTANCE CRITERIA	RATIONALE
FR-001	Functional	The system shall ping microservice health endpoints at 30-second intervals and compute rolling 24-hour service availability percentages.	High	Pass: Downtime is recorded and an alert is dispatched after 3 consecutive failed health probes. Fail: An unreachable service is marked as healthy.	Ensures continuous monitoring of microservice availability and timely detection of service failures.
FR-002	Functional	The system shall allow a DevOps Engineer to register a microservice by providing its name, health endpoint, and required service details.	High	Pass: A microservice with valid details is successfully added to the service catalog. Fail: A microservice with missing mandatory details is registered.	Provides the service information required for cataloging and health monitoring.
FR-003	Functional	The system shall display the dependencies between registered microservices as an interactive dependency graph .	High	Pass: Selecting a registered microservice displays its dependencies and dependent services in the graph. Fail: A known dependency is missing from the displayed graph.	Enables users to understand the relationships and architecture of the microservices.
FR-004	Functional	The system shall allow a System Architect to view API documentation associated with registered microservices through the portal.	Medium	Pass: Selecting a registered microservice displays its available API documentation. Fail: Available API documentation cannot be accessed for the selected microservice.	Centralizes API documentation and provides convenient access to service information.

REQ ID	TYPE	DESCRIPTION	PRIORITY	ACCEPTANCE CRITERIA	RATIONALE
FR-005	Functional	The system shall allow a DevOps Engineer to configure the health check interval and consecutive-failure threshold for a registered microservice.	High	Pass: Updated interval is applied starting from the next scheduled health check. Fail: Health checks continue running on previous interval after a change is saved.	Allows monitoring sensitivity to be tuned per service without requiring code changes.
NFR-001	Nonfunctional	The service catalog dependency graph viewer must render interactive node architectures containing up to 200 services smoothly.	High	Pass: Benchmarking tests confirm that the dependency graph meets the required performance and security standards under simulated peak load. Fail: The dependency graph fails to meet the required performance or security standards under simulated peak load.	Ensures that the dependency graph remains usable for large microservice architectures.
NFR-002	Nonfunctional	The system shall restrict access to microservice catalog and health-monitoring functions to authenticated users.	High	Pass: An unauthenticated user attempting to access protected functions is denied access. Fail: An unauthenticated user can access protected microservice information or monitoring functions.	Protects internal microservice architecture and operational information from unauthorized access.

2. Use case diagram



3. Use-Case Flow Document

Use Case: Monitor Service Health

Actors: DevOps Engineer (primary), Alerting System (secondary)

Preconditions:
The microservice is registered in the catalog with a configured health check endpoint and polling interval. The health check scheduler is active and the DevOps Engineer is logged in.

Postconditions:
The service's rolling 24-hour availability percentage is updated and visible on the dashboard. If downtime was detected, an alert has been dispatched to the Alerting System and logged against the service.

Main Success Scenario:

1. DevOps Engineer opens the Monitor Service Health dashboard for a registered microservice.
2. System retrieves configured health check interval (e.g. 30 seconds) for the service.
3. System pings the service's health endpoint at configured interval (includes Ping Health Endpoints).
4. Service responds with a healthy status within expected timeout.
5. System records successful probe result with a timestamp.
6. System recomputes rolling 24-hour availability percentage for service.
7. System updates dashboard with current status and availability percentage.
8. Use case ends successfully.

Alternate Flow:

4a. Health Probe Fails

4a1. Service does not respond, or responds with an error status, within timeout window.
4a2. System logs failed probe with a timestamp.
4a3. System checks count of consecutive failed probes for service.
4a4. If fewer than 3 consecutive failures, the system returns to step 3 at next scheduled interval.
4a5. If 3 consecutive failures are reached, system marks service as down and triggers Dispatch Downtime Alert (extend).
4a6. System sends alert to the Alerting System with the service name, status, and failure timestamp.
4a7. DevOps Engineer is notified and can view the incident on the dashboard. Use case ends.